In this chapter, we'll look at Ruby's different threading abstractions that allow you to organize your program so that you can run different parts of it apparently "at the same time." The Thread class is the basic unit of multithreaded behavior in Ruby. Ruby also allows you to spawn processes out to the underlying operating system and multithread those processes. *Fibers* are an additional abstraction that lets you suspend execution of one part of your program and run some other part. Finally, The Ractor library allows you to bypass the GIL and have true multithreading using Ruby.

Let's start with the Thread class, which is the basis for Ruby multithreaded behavior.

Multithreading with Threads

The lowest-level mechanism in Ruby for doing two things at once is to use the Thread class. Although threads can in theory take advantage of multiple processors or multiple cores in a single processor, there's a major catch. Many Ruby extension libraries are not thread safe, because they expect to be protected by the GIL. So, Ruby uses native operating system threads but operates only a single thread at a time. Unless you use the Ractor library, you'll never see two threads in the same application running Ruby code truly concurrently. You will, however, see threads busy executing Ruby code while another thread waits on an I/O operation.

Creating Ruby Threads

The code that follows is a simple example. It downloads a set of web pages in parallel. For each URL that it is asked to download, the code creates a separate thread that handles the HTTP transaction:

tut_threads/fetcher.rb

```
require "net/http"

pages = %w[www.rubycentral.org www.pragprog.com www.google.com]

threads = pages.map do |page_to_fetch|
  Thread.new(page_to_fetch) do |url|
    http = Net::HTTP.new(url, 80)
    print "Fetching: #{url}\n"
    response = http.get("/")
    print "Got #{url}:  #{response.message}\n"
  end
end
threads.each { |thread| thread.join }
print "We're done here!\n"
```

Which results in:

```
Fetching: www.rubycentral.org
Fetching: www.pragprog.com
Fetching: www.google.com
Got www.google.com:  OK
Got www.pragprog.com:  Moved Permanently
Got www.rubycentral.org:  OK
We're done here!
```

Let's look at this code in more detail, because a few subtle things are happening.